Design Proposal — Archive → RAG Collection Ingestion Framework

Status: proposal · Audience: archive stakeholders + engine implementers

> **AI usage acknowledgement.** This design document was produced collaboratively with Claude (Anthropic). The author directed the work — setting the goals, parameters, and constraints discussed with the BKC team — and Claude assisted by exploring the codebase, drafting the proposal, and refining it under the author's review. All decisions and final content are the author's.

Context

The BKC Archive Wiki is a plain-markdown knowledge base (`archive/archive-wiki/`) built over an immutable source (`raw/archive.json`, 6,925 bookmarks). Separately, this engine (`llm_engine/`) runs a LangChain + Chroma RAG stack (OpenAI embeddings) whose `src/agents/helpers/rag.ts` already exposes the primitives we need: `addTextsToVectorStore`, `addPDFToVectorStore`, `getContextChunksForQuestion`, and collection CRUD. A one-off script, `scripts/loadArchiveIntoTopic.ts`, already chunks the wiki's `.md` bodies into a Chroma collection with citation metadata.

What's missing is a **general, repeatable framework** that ingests a *whole archive collection* — not just markdown, but the raw JSON item metadata, txt transcripts, PDFs, and future textual formats — into RAG through one parser-driven pipeline. This document is the design for that framework.

Decisions confirmed with the owner: audience is **hybrid** (stakeholder brief + implementer detail); the collection is a **sibling input directory feeding the wiki**; the vector store is **one shared Chroma collection, filtered by metadata**.

---

1. Executive brief (for stakeholders)

We will give the archive a single, well-defined "front door" into the RAG system. A **collection** is a folder on disk holding everything about one body of archive material: the human-readable wiki pages *and* the raw source items they're built from (JSON metadata, transcripts, PDFs, and other text). A small **ingestion framework** walks that folder, recognizes each file by type, runs the right **parser** to turn it into clean text plus structured **metadata**, and loads it into the existing vector store. The LLM can then answer questions over the archive and **cite back** to the exact item — by title, source, URL, and date.

The design's purpose is to make ingestion **format-agnostic, idempotent, and extensible**: adding a new file type (e.g. `.docx`, `.csv`) means registering one new parser, not rewriting the pipeline; re-running ingestion updates rather than duplicates; and every retrieved chunk carries enough provenance to be cited and trusted. It reuses the engine that already exists rather than building a parallel one.

---

2. Collection folder structure (the input contract)

A **collection** is a self-describing folder. The framework reads it; the wiki is one *view* of it. Proposed layout (sibling to / containing the existing wiki):

```
collection/                      one archive collection
├── collection.json              manifest: id, title, description, defaults
├── wiki/                         LLM-maintained synthesis (markdown)
│   ├── items/<year>/<id>-<slug>.md
│   ├── events/<year>/<id>-<slug>.md        most items are events …
│   ├── publications/<year>/<id>-<slug>.md  … or publications
│   ├── topics/  people/  orgs/  sources/  timeline/
│   └── index.md  log.md
└── raw/                          immutable source items, by format
    ├── json/   item metadata (archive.json array, or per-item *.json)
    ├── txt/    transcripts / plain-text source bodies
    ├── pdf/    full-text documents
    └── <other>/  future formats (one subfolder per type is optional)
```

- `wiki/` is the current `archive-wiki/` tree, now including `events/` and `publications/` — the two categories most archive items fall into (events is already added to the wiki; the collection is not yet fully populated).
- `raw/` is the new part: the actual source contents, segregated by format so a parser registry can map a subfolder/extension → loader.
- `collection.json` carries collection-level defaults (collection id used in metadata, embeddings model override, chunk sizing, include/exclude globs) so the same code serves many collections with no edits.

This generalizes today's split (script-owned generated layer vs. LLM-owned synthesis) by adding a third, explicit **raw-source layer** that RAG can index directly.

---

3. Architecture (for implementers)

One pipeline, three stages: **discover → parse/normalize → load**.

```
walk(collection)  →  Parser registry (by ext)  →  IngestRecord[]  →  load
into Chroma
                       .md  → markdown parser        { text,
(rag.addTextsToVectorStore
                       .json→ json item parser         metadata }       or
rag.addPDFToVectorStore)
                       .txt → text parser
                       .pdf → pdf parser
                       …    → (register more)
```

3.1 Parser registry

A map `extension → parse(file, collectionDefaults) → IngestRecord[]`. Each parser owns its format and emits a normalized record shape; the driver stays format-blind.

- **markdown** — reuse the frontmatter-stripping `parse()` already in `loadArchiveIntoTopic.ts`; body → `text`, frontmatter → metadata.
- **json** — handle both the bulk `archive.json` (`.items[]`) and per-item files; synthesize `text` from `title + description + content` and map `title/url/date_published/tags` into metadata. One record per item.
- **txt** — whole file → `text`; metadata from filename + optional `<name>.meta.json` sidecar.
- **pdf** — route to existing `addPDFToVectorStore` (uses `PDFLoader`), which already has a "is this PDF already indexed?" guard.

3.2 Normalized record + metadata schema (the citation contract)

Every text chunk lands in the **one shared** collection, so metadata *is* the namespace and the provenance. Proposed fields on each record's `metadata`:

field	purpose
`collectionId`	which archive collection (filter key for the shared store)
`archiveType`	the kind of archive item: `item` | `event` | `publication` | `topic` | `person` | `org` | `timeline` | `source` | `raw`. Most archive items are an `event` or a `publication`; `item` covers anything not yet sorted into one of those categories
`format`	`markdown` | `json` | `txt` | `pdf` | …
`conversationName`	human-facing **source label** (= title) — kept for retrieval/citation compatibility with the existing historian tool
`title`, `url`, `source`, `date`	provenance for citations
`sourceId`	stable id (item id / file path) for dedup + targeted deletion
`contentHash`	sha256 of chunk text → idempotency (resolves the `// TODO use Hash` in rag.ts)

3.3 Load + idempotency

- Text records → `rag.addTextsToVectorStore(SHARED_COLLECTION, texts, { metadatas })`.
- PDFs → `rag.addPDFToVectorStore(...)`.
- Idempotency: compute `contentHash` per chunk; skip chunks already present (query by `{sourceId}` / `{contentHash}` filter) or `removeFromVectorStore({sourceId})` then re-add on change. This makes re-ingestion safe — the framework's "build" equivalent.

3.4 Retrieval (unchanged surface, new affordance)

`getContextChunksForQuestion` already accepts a `filter`. Because everything is in one collection, callers scope a query by passing `filter: { collectionId, archiveType, … }`. No new retrieval code is required — only that we *populate* the filterable metadata.

---

4. Key parameters & affordances

- **Shared collection name** — single constant (overridable via `collection.json`).
- **Embeddings model** — defaults to the engine's document model; per-collection override passes through `addTextsToVectorStore`'s `embeddingsModelName`.
- **Chunking** — defaults `chunkSize 1000 / overlap 200` (today's values); allow per-format override (e.g. larger for PDFs) in the manifest.
- **Metadata filters** — the levers that separate archives/types in one store.
- **Idempotent re-runs** — `contentHash` dedup; `--clean` to rebuild.
- **Extensibility** — new format = one registry entry; no driver changes.
- **Operational flags** — `--collection <path>`, `--limit N`, `--dry-run`, `--only <format>`, mirroring `loadArchiveIntoTopic.ts`'s ergonomics.

---

5. Implementation sketch (where code lands)

Build on what exists; do not fork the RAG layer.

- **New:** `src/ingest/parsers/` — one module per format (`markdown.ts`, `json.ts`, `txt.ts`, `pdf.ts`) plus a `registry.ts`.
- **New:** `src/ingest/ingestCollection.ts` — the discover→parse→load driver, emitting `IngestRecord[]` and calling `rag.ts`.
- **New (thin):** `scripts/ingestCollection.ts` — CLI wrapper, generalizing `loadArchiveIntoTopic.ts` (which becomes a special case / can be deprecated).
- **Reuse:** all of `src/agents/helpers/rag.ts`; the markdown `parse()` and `walk()` helpers from `loadArchiveIntoTopic.ts`.
- **Optional small change to rag.ts:** wire `contentHash` dedup into `addTextsToVectorStore` (the existing TODO) so idempotency is centralized.

---

6. Verification

1. **Unit** — each parser: feed a fixture file, assert `IngestRecord[]` text + metadata (esp. citation fields) — sits alongside existing `tests/`.
2. **Idempotency** — ingest a small `collection/` twice; assert chunk count is stable (no duplicates) via `rag.checkCollections()`.
3. **End-to-end** — run `node scripts/ingestCollection.ts --collection <path> --limit 20` against a mixed md/json/txt/pdf fixture, then call `getContextChunksForQuestion` with a metadata filter and confirm chunks return with correct `title/url/source`.
4. **Citation** — confirm a retrieved PDF/JSON chunk surfaces a usable `conversationName`/`url` so the agent can cite it.

---

7. Open questions (flag, don't block)

- Does `raw/` get committed to git, or is large binary/PDF content stored out-of-band (LFS / object store) with only manifests in git?
- Should `collection.json` be authored by hand or generated by the wiki's `build.mjs` so the two layers stay in sync?

- Confirm whether `loadArchiveIntoTopic.ts`'s per-topic collection usage must keep working during/after the move to one shared collection.